School of Future Tech

Case Study Report

on

Case Study 123: Weather Forecast Simulation (C++)

by

Ankitraj Jha

150096725050

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References

1. Introduction to the Case Study

Weather forecasting systems require efficient data structuring and retrieval mechanisms to provide accurate and timely information to users. This case study focuses on designing and implementing a Weather Forecast Simulation using C++. The system efficiently manages and displays weather conditions (such as temperature and humidity) for various cities using predefined datasets.

It highlights the importance of data representation using object-oriented principles, as well as the application of arrays and conditional statements to simulate real-world information systems, proving fundamental to software engineering.

2. Problem Statement / Case Background (Abstract)

Background

Traditional manual data lookups are slow and highly prone to human error. Developing automated, menu-driven software systems to handle structured lists of data ensures accuracy and provides a better user experience**.**

Abstract

This case study presents the design and implementation of a **Weather Forecast Simulation in C++**. A predefined dataset containing city names, temperatures, and humidity levels is structured using Object-Oriented Programming (OOP) concepts. The program implements a dynamic, menu-driven interface to perform operations such as displaying the weather for a selected city, comparing weather conditions between two cities side-by-side, and calculating global weather statistics like average, highest, and lowest temperatures. The system effectively utilizes arrays for data holding, conditional logic for selective data retrieval, and precise output formatting for a clean user interface.

3. Case Study Design

The Weather Forecast Simulation system is designed as a modular program with the following core components:

Data Representation

- Design a `WeatherData` class to encapsulate city-specific properties: city name, temperature, and humidity.

- Implement read-only getter methods for clean and secure data access.

Array-based Data Storage

- Utilize an array of objects to store the predefined database of 5 cities seamlessly in memory.

Core Application Logic and Conditional Routing

- Implement a menu-driven simulation loop allowing the user to dynamically view cities, sequentially check specific weather combinations, compare cities, or view global application statistics.

Mathematical Analysis

- Implement a `WeatherAnalyzer` class to iterate through the array properties to determine the average, maximum, and minimum recorded temperatures across all cities.

Formatting and Structured Output

- Use the C++ library output manipulators to render tabular, boxed text interfaces for visually appealing reports.

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

Key methods and algorithms:

Object-Oriented Principles

Encapsulation: Keeping weather data variables private and safely retrieving them.

Constructors_: Parameterized constructors to quickly instantiate populated city objects upon application launch.

Array Iteration & Linear Search

Linear search iterations to scan arrays to identify specific extremes, such as determining the hottest and coldest indexes safely.

Accumulation algorithms to dynamically compute aggregate mathematical averages across datasets.

Conditional Logic & Fault Tolerance

Rigid input validation checking (intercepting string mismatches) to ensure robust execution and prevent infinite system loops.

Implementation of flow control statements (if/else, switch-case) for user-driven menu routing.

Technology Stack Used for the Case

Programming language: C++

Libraries and tools:

- `<iostream>` for core standard input and output stream operations.
- `<string>` for dynamic text management of city names.
- `<iomanip>` for complex, precise terminal output formatting operations (`setw`, `setprecision`, `fixed`, `left`, `right`).

Environment : Built for cross-platform compatibility across standard C++ compilers (GCC, Clang, MSVC) via IDEs like VS Code.

5. Problem Statement / Case Study Implementation Details and Snapshots.

The implementation is strategically organized in a single C++ source file (`WeatherForecast.cpp`) and logically modularized into distinct classes:

WeatherData Class

Handles the individual city profiles and the localized, console-drawn formatted printing of single-city weather traits.

WeatherAnalyzer Class

A static functional utility that ingests the full city dataset array and generates aggregated global calculations.

WeatherForecastSimulation Class

The foundational execution driver handles the primary array storage matrix, processes continuous user inputs via loops, handles indexing formulas for city comparison logic, and acts as the application's main thread.

Core Code Snippet:

```
class WeatherData {

private:
```

```
 string cityName;
 double temperature;
 double humidity;
public:
 WeatherData() : cityName(""), temperature(0.0), humidity(0.0) {}
 WeatherData(string city, double temp, double hum) {
   cityName = city;
   temperature = temp;
   humidity = hum;
 }
 string getCityName() const { return cityName; }
 double getTemperature() const { return temperature; }
 double getHumidity() const { return humidity; }
 void displayFormattedOutput() const {
   cout << "\n+--------------------------------------+" << endl;
   cout << "|             WEATHER REPORT            |" << endl;
   cout << "+--------------------------------------+" << endl;
   cout << "| City       : " << setw(24) << left << cityName << "|" << endl;
   cout << "| Temperature: " << fixed << setprecision(1) << temperature
        << "\xC2\xB0" << "C" << setw(18) << right << "|" << endl;
   cout << "| Humidity   : " << fixed << setprecision(1) << humidity << "%"
        << setw(19) << right << "|" << endl;
   cout << "+--------------------------------------+" << endl;
 }
};
```

Snapshots :

- **Main Menu**

```
==========================================
        WEATHER FORECAST SIMULATION
==========================================
 1. View Cities
 2. Check Weather
 3. Compare Two Cities
 4. View Global Weather Statistics
 5. Exit
==========================================

=> Enter choice: 1

+----------------------------------------+
|              View Cities               |
+----------------------------------------+
|  1. Mumbai                             |
|  2. Delhi                              |
|  3. Bangalore                          |
|  4. Chennai                            |
|  5. Kolkata                            |
+----------------------------------------+
```

- **Check Weather**

```
==========================================
        WEATHER FORECAST SIMULATION
==========================================
 1. View Cities
 2. Check Weather
 3. Compare Two Cities
 4. View Global Weather Statistics
 5. Exit
==========================================

=> Enter choice: 2

+----------------------------------------+
|              View Cities               |
+----------------------------------------+
|  1. Mumbai                             |
|  2. Delhi                              |
|  3. Bangalore                          |
|  4. Chennai                            |
|  5. Kolkata                            |
+----------------------------------------+

=> Select city (1-5): 3

+----------------------------------------+
|             WEATHER REPORT             |
+----------------------------------------+
| City        : Bangalore                |
| Temperature: 24.0°C                    |
| Humidity    : 60.0%                    |
+----------------------------------------+
```

- **Compare Two Cities**

```
==========================================
       WEATHER FORECAST SIMULATION
==========================================
 1. View Cities
 2. Check Weather
 3. Compare Two Cities
 4. View Global Weather Statistics
 5. Exit
==========================================

=> Enter choice: 3

+----------------------------------------+
|              View Cities               |
+----------------------------------------+
|  1. Mumbai                             |
|  2. Delhi                              |
|  3. Bangalore                          |
|  4. Chennai                            |
|  5. Kolkata                            |
+----------------------------------------+

=> Select first city  (1-5): 1
=> Select second city (1-5): 4

+-------------------------------------------------------+
|                WEATHER COMPARISON                     |
+--------------+--------------------+----------------+
| Detail       | Mumbai             | Chennai        |
+--------------+--------------------+----------------+
| Temperature  | 32.0°C             | 34.0°C         |
| Humidity     | 70.0%              | 75.0%          |
+--------------+--------------------+----------------+
```

- **View Global Weather Statistics**

```
==========================================
       WEATHER FORECAST SIMULATION
==========================================
 1. View Cities
 2. Check Weather
 3. Compare Two Cities
 4. View Global Weather Statistics
 5. Exit
==========================================

=> Enter choice: 4

+----------------------------------------+
|       GLOBAL WEATHER STATISTICS        |
+----------------------------------------+
| Average Temp :                  29.9°C |
| Hottest City :                 Chennai |
| Coldest City :               Bangalore |
+----------------------------------------+
```

- **Exit From the Simulation**

```
==========================================
       WEATHER FORECAST SIMULATION
==========================================
 1. View Cities
 2. Check Weather
 3. Compare Two Cities
 4. View Global Weather Statistics
 5. Exit
==========================================

=> Enter choice: 5

Exiting Weather Forecast Simulation. Goodbye!
```

6. Problem Statement / Case Study Results and Conclusion.

Findings

- Utilizing strict Class boundary logic allows for code that is robust, heavily modularised, and easy to maintain or scale if the city array density needed to be massively increased.
- Input validation is critical in CLI programs to avoid array out-of-bounds execution errors or stream failure loops.
- The C++ `<iomanip>` feature-set gives immense control over layout construction, converting raw text strings into professional visual tables.

Conclusion

This case study demonstrates a successful pipeline for building a Weather Forecast Simulation using C++. It integrates robust data representation through OOP, effective memory array management to simulate local databases, complex conditional operations to handle unpredictable user state input, and comprehensive output logic. The system successfully executes all primary objectives set out by the academic requirements, providing a resilient educational foundation in software system structuring.

7. References

- C++ Standard Library Reference: en.cppreference.com (Core IO and String algorithms).
- LearnCpp.com: Comprehensive C++ Programming Tutorials (Object-Oriented
- From Lecture Notes.